1 Lesson 23-10-26

So far we introduced a number of processor architecture aimed at lowering **CPI** (*Cycles Per Instruction*).

- One example was the **pipeline**, which leveraged **ILP** (*Instruction-Level Parallelism*) to increase the number of instructions taken into the execution unit at each cycle;
- We then noticed pipelines benefitted for some reordering of instructions, aimed to reduce hazards preserving dependencies, called **scheduling**. We saw schedulilng of either the static (think IA64) or the dynamic (*Tomasulo*) type;
- We then extended the Tomasulo architecture, by introducing the **ROB** (*ReOrder Buffer*), and thus educing the effects of data and control dependencies (by allowing dynamic branch prediction, and *speculation* acting on those predictions).

We know want to focus on *superscalar* architectures, able to finish more than one instruction at each clock cycle.

1.1 Multiple-issue processors

The techniques for reducing the effects of data and control dependencies can be further exploited to obtain a CPI less than one: this requires issuing more than one instruction per clock cycle. There are two kinds of processors able to do so:

- **Superscalar** processors, either statically or dynamically scheduling instructions;
- *Very Long Instruction Word* (**VLIW**) processors.

In both groups the architecture obviously includes multiple functional units.

1.1.1 Static multiple issue

Implementing a superscalar processor able to possibly issue several instructions according to the static order defined by the compiler is rather easy and inexpensive.

For instance, two instructions can be issued per clock cycle if:

- One is a load, store, branch, or integer ALU operation;
- The other is any FP operation (but load and store, which belong to the first category).

Two instructions (64 bits) are fetched and decoded at every clock cycle. The two instructions are aligned on a 64-bit boundary and constitute an *issue packet*.

Old superscalar processors required a fixed structure for issue packets (e.g., that the integer instruction is always the first one). Current processors normally do not have this limitation.

1.1.2 Dynamic multiple issue

In dynamic multiple issue, it's the processor's job to decide how to schedule instructions. Multiple issue in this case just means taking the modified Tomasulo architecture and allowing for multiple instructions to be dispatched at once.

Similarly, at this point, the commit unit will be able to take more than one instruction out of the reorder buffer at any given cycle, and commit them together.

One complicaton emerges from branches: in generale, whether we allow or not for speculation, it is best to have branch instruction dispatched alone at any given cycle.

1.2 Caches

Modern processors heavily rely on **caches** to speed up execution. Caches exploit *locality* of data and code to vastly reduce the time losses due to memory accesses.

At the most general level:

- If an entire block containing a referenced memory location is loaded in cache at time t_0;
- The program will likely find the rest of the memory it needs within the block for a time Δt.

There are 2 types of locality:

- **Temporal** locality: the same data is used more often if it was recently referenced. Within our previous framework, if at time t the progarm accesses a block of memory, it will likely access it again until $t + \Delta t$;
- **Spatial** locality: data around the data which was just used is more likely to be used next. Within our previous framework, if at time t the program accesses a block of memory at address x, it will likely also access memory at $x \pm \Delta x$.

Cache performance is given by the average *access time* in function of the hit rate h:

$$t_{ave} = h \times C + (1 - h) \times M$$

where C is the cache access time (on cache *hit*), and M is the memory access time (on cache *miss*).